MCMP117 · WEEK 15

Fundamentals of Operating Systems

Week 15 · Final Exam Review & Practice

Fifteen weeks, one exam: from the first batch systems to your own shell scripts.

Electronical Communication Technology Programme · Vocational School · Istanbul Okan University

Exam Logistics

When and where

[Date, time and room: to be announced on the course Telegram channel]

Duration

[90] minutes. Arrive [15] minutes early; seating as announced.

Paper

100 points in six parts (A–F), exactly like the sample paper you receive today.

Bring

Student ID, pens. [Basic, non-programmable calculator: allowed — to be confirmed]

Rules

Closed book [confirm]. [A formula sheet is / is not provided — to be confirmed]. Phones off, in your bag.

Weight

[The final counts __ % of the course grade — see the syllabus / to be confirmed]

Anything in [square brackets] is not yet official. The final announcement on Telegram is what counts.

FINAL EXAM

What the Final Covers: the Whole Course

Part	Question type	Weeks	Points*
A	Multiple choice: concepts and commands	W1–W7, W9–W14	20
B	Short answers: explain, compare, list	W1–W7	12
C	Numerical problems: Gantt chart, paging/TLB, page replacement, EAT, block arithmetic	W4–W7 (+ W1, W3)	36
D	Write the command	W9–W13	14
E	Write the pipeline	W12	9
F	Write a short Bash script	W14	9

*Point split of the sample paper [the real paper may differ slightly — confirm]. Week 8 (midterm) adds no new material; its study guide is still valid for Weeks 1–7.

Today's Agenda

1. **Theory recap** Weeks 1–7 in seven slides, plus a one-page formula sheet.
2. **Theory practice** Four worked problems: scheduling, paging + TLB, page replacement, EAT and blocks.
3. **Practical recap** Weeks 9–14: shell, files, permissions, pipes, processes, scripts — real output.
4. **Practical practice** Write the command, the pipeline, the script: three worked problems.
5. **Exam strategy** Common mistakes, how to use the 90 minutes, a quick quiz.
6. **Course wrap-up** What you have learned, and where to go next.

PART 1

Theory Recap: Weeks 1–7

Same terms, same formulas, same conventions as in the weekly notes.

History, Definitions, Goals and Structures

Week 1 — Why operating systems exist

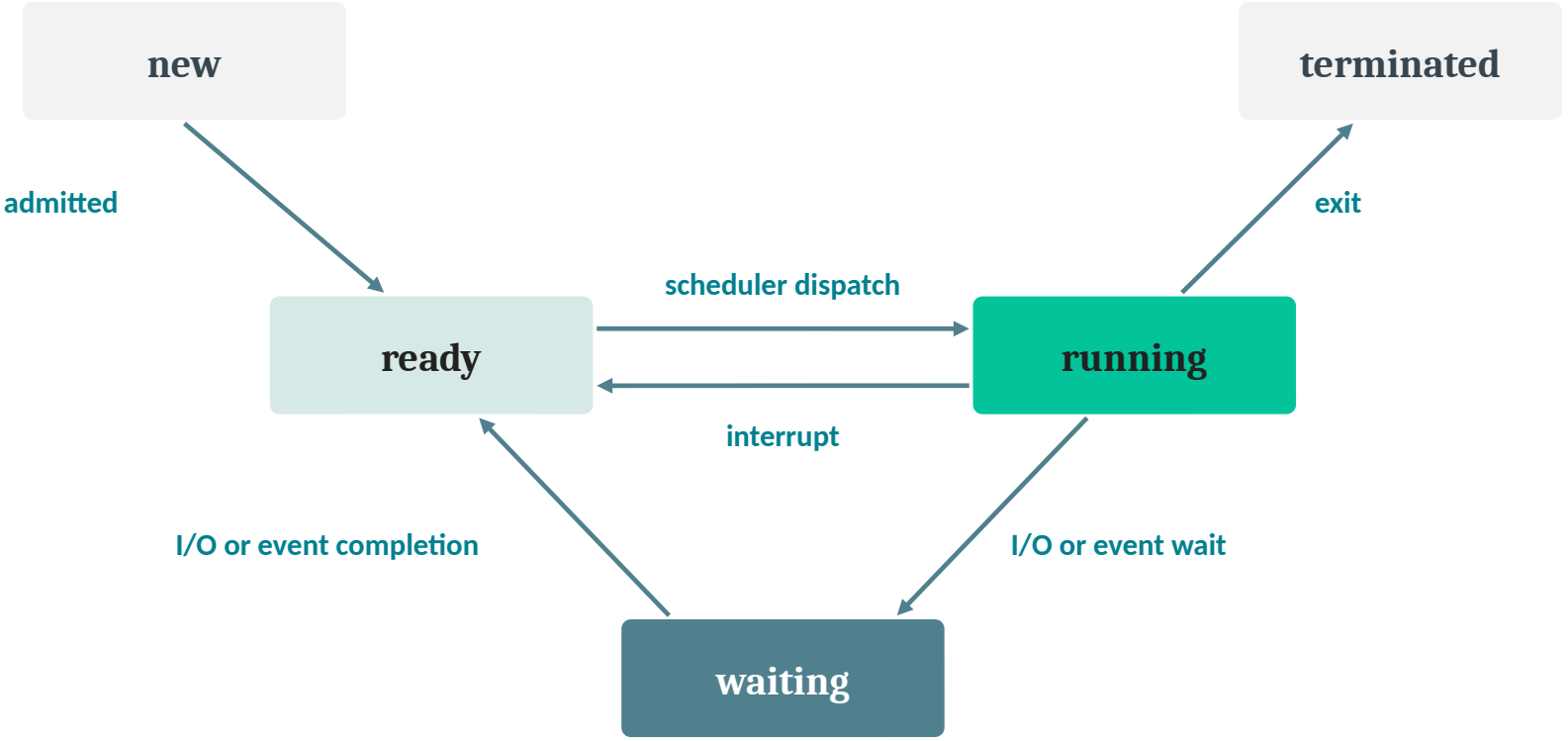
- No OS (ENIAC, 1945) → batch + resident monitor (GM-NAA I/O, 1956) → multiprogramming and time-sharing (1960s) → PCs → phones and cloud.
- **Idle CPU drove everything:** $U = t_{run} / (t_{run} + t_{setup})$; with n jobs waiting for I/O a fraction p of the time: $U(n) = 1 - p^n$.
- **Dates:** UNIX 1969 (Bell Labs), Linux 1991 (Torvalds).

Week 2 — What an OS is

- **Two views:** extended machine (abstraction) and resource manager (time and space multiplexing). Kernel = always-resident core.
- **Goals:** convenience, efficiency, ability to evolve.
- **Hardware:** interrupts; storage hierarchy registers → cache → RAM → SSD → HDD.
- **Dual mode:** mode bit 0 = kernel, 1 = user; a system call traps into kernel mode.
- **Structures:** simple, monolithic, layered, microkernel, modular, hybrid.

Typical questions: compute U for a batch or multiprogrammed system; explain the two views; why dual mode needs hardware; monolithic vs. microkernel (speed vs. reliability).

Services, System Calls and the Process Life Cycle



Never waiting → running directly: a woken process joins the ready queue first. ps STAT: R, S, D, T, Z (Week 13).

Also from Week 3

- **System call:** API → wrapper → trap; number indexes the system-call table.
- **PCB:** state, PC, registers, scheduling, memory, files.
- **Context switch:** save one PCB, load another; overhead $s/(q + s)$.
- **fork():** n forks in a loop → 2^n processes ($2^n - 1$ created).

Threads and Scheduling Criteria

Threads

- **Own:** thread ID, program counter, registers, stack.
- **Shared:** code, data, heap, open files of the process.
- **Models:** many-to-one, one-to-one (Linux, Windows), many-to-many.
- Concurrency ≠ parallelism (parallelism needs several cores).

Amdahl's law

- **speedup ≤ 1 / (S + (1 − S)/N)**, limit 1/S.
- Example S = 0.2, N = 4: 1/(0.2 + 0.2) = 2.5; never more than 5.

```
# Scheduling criteria (Week 4)
T_turnaround = T_completion − T_arrival
T_waiting    = T_turnaround − T_burst
T_response   = T_first run  − T_arrival

CPU utilisation = B / T     (busy / total)
throughput      = n / T     (jobs per time)
```

Goal

Maximise utilisation and throughput; minimise turnaround, waiting and response time. The scheduler mainly changes the waiting time.

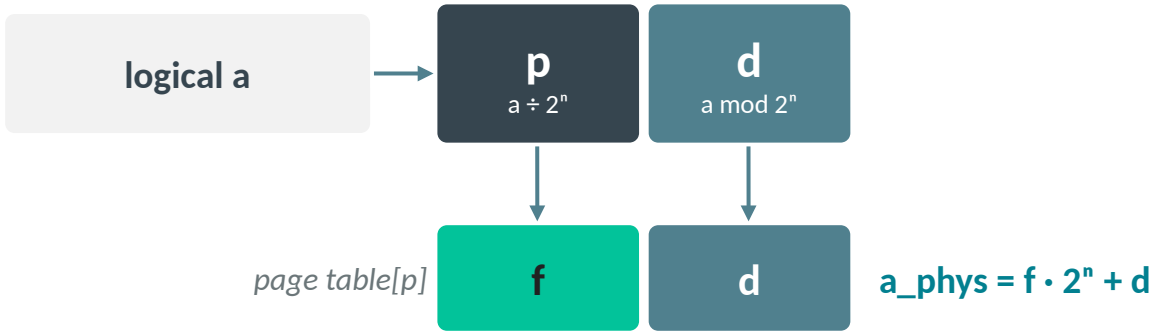
Scheduling Algorithms and Synchronisation

Algorithm	Picks	Preemptive?	Watch out for
FCFS	earliest arrival	no	convoy effect
SJF	shortest next burst	no	needs burst prediction $\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$
SRTF	shortest remaining time	yes	minimum average wait, but long jobs may starve
Priority	smallest number = highest priority	either	starvation → aging
Round Robin	head of the queue, at most q	yes	wait between turns ≤ $(n - 1)q$

Synchronisation

- **Race condition:** shared data, at least one writer, result depends on timing (counter++ = load–add–store).
- **Critical-section solution:** mutual exclusion, progress, bounded waiting. Tools: mutex (acquire/release), semaphores wait(S) / signal(S); bounded buffer with empty, full, mutex.

Memory Management: From Base/Limit to Paging



Before paging

- **MMU:** if 0 ≤ a < limit then a_phys = base + a, else trap.
- **Fits:** first (first hole big enough), best (smallest), worst (largest).
- **Fragmentation:** external (holes; 50-percent rule ⇒ up to 1/3 lost), internal (inside a block; paging: ½ page per process).

TLB (our convention)

- **EAT = t_TLB + (2 − α) t_m**
- hit: t_TLB + t_m; miss: t_TLB + 2t_m (page table + data).
- Simplified (t_TLB = 0): α t_m + (1 − α) 2t_m — say which one you use.

Segmentation

- Address ⟨s, d⟩; if 0 ≤ d < limit[s] then a_phys = base[s] + d, else trap.
- Page table size = number of pages × entry size (2^{20} × 4 B = 4 MB).

WEEK 7

Virtual Memory and File Systems

Virtual memory

- Demand paging: load a page on first use; valid–invalid bit; page fault = trap, load, restart instruction.
- **EAT = (1 − p) t_m + p t_fault** — p must be tiny (≈ 10^{-6}).
- Copy-on-write makes fork() cheap.
- **Replacement:** FIFO (Belady's anomaly possible), OPT (needs the future), LRU (clock approximates it).
- Thrashing: frames < working set → paging instead of computing.

File systems

- File = named data + attributes (Linux: inode); open() once, then use the file descriptor.
- **Access:** sequential, direct (block = ⌊offset / block size⌋), indexed.
- **Directories:** single-level, two-level, tree, acyclic graph, general graph.
- **Links:** hard = another name for the same inode; symbolic = small file holding a path (can dangle).

Typical questions: frame tables for FIFO/OPT/LRU, page-fault EAT and the largest p for a given slow-down, block number and offset of a record, hard vs. symbolic link after rm.

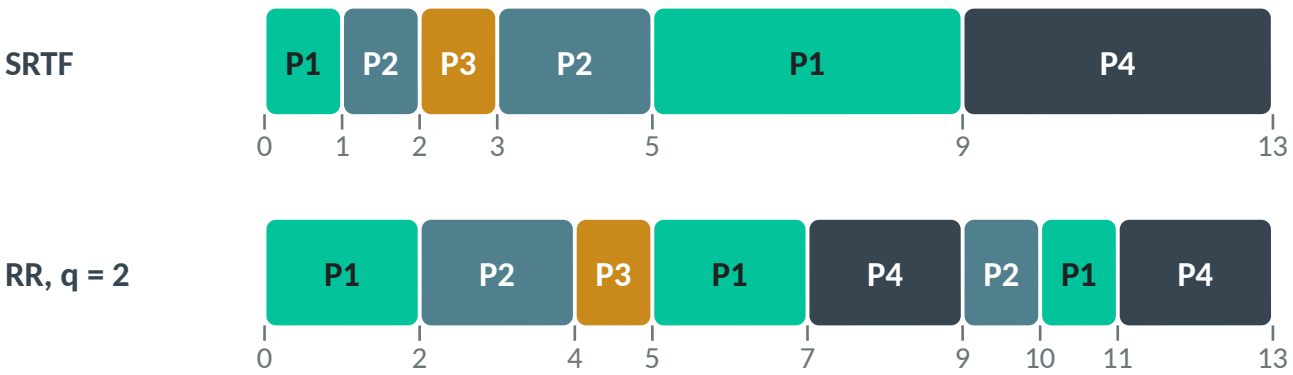
THEORY

The Formula Sheet on One Slide

Week	Quantity	Formula
W1	Batch utilisation	$U = t_{run} / (t_{run} + t_{setup})$
W1	Multiprogramming	$U(n) = 1 - p^n$
W3	Switch overhead	$s / (q + s)$; n forks $\rightarrow 2^n$ processes
W4	Amdahl	$\text{speedup} \le 1 / (S + (1 - S)/N)$
W4	Times	$TAT = \text{done} - \text{arrival}$; $\text{wait} = TAT - \text{burst}$; $\text{resp} = \text{first run} - \text{arrival}$
W5	Burst prediction	$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$; RR wait $\le (n - 1)\, q$
W6	Paging	$p = \lfloor a / 2^n \rfloor$, $d = a \bmod 2^n$, $a_{phys} = f \cdot 2^n + d$
W6	TLB	$EAT = t_{TLB} + (2 - \alpha)\, t_m$
W7	Page faults	$EAT = (1 - p)\, t_m + p\, t_{fault}$
W7	Direct access	$\text{byte} = r \times \text{size}$; $\text{block} = \lfloor \text{byte} / B \rfloor$; $\text{offset} = \text{byte} \bmod B$

SRTF and Round Robin (q = 2) on Four Processes

P1: arrives 0, burst 5 P2: arrives 1, burst 3 P3: arrives 2, burst 1 P4: arrives 3, burst 4 (ms)



SRTF

	Arr.	Burst	Finish	Turnar.	Wait	Resp.
P1	0	5	9	9	4	0
P2	1	3	5	4	1	0
P3	2	1	3	1	0	0
P4	3	4	13	10	6	6
Avg.				**6**	**2.75**	**1.5**

RR, q = 2

	Arr.	Burst	Finish	Turnar.	Wait	Resp.
P1	0	5	11	11	6	0
P2	1	3	10	9	6	1
P3	2	1	5	3	2	2
P4	3	4	13	10	6	4
Avg.				**8.25**	**5**	**1.75**

Read the result

- Average waiting: SRTF 2.75 ms, RR 5 ms.
- Average response: SRTF 1.5 ms, RR 1.75 ms.
- SRTF minimises waiting; RR gives every process a turn quickly.

Address Translation and TLB Effective Access Time

The question

- Pages of 1 KB. Page table: page 0 → frame 5, 1 → 2, 2 → 7, 3 → 1.
- (a) Translate logical address 2500.
- (b) t_TLB = 10 ns, t_m = 100 ns, α = 0.9: EAT? Which α gives 115 ns?

```
# (a) 1 KB = 2^10 = 1024 bytes
p = ⌊2500 / 1024⌋ = 2
d = 2500 − 2 × 1024 = 452
page 2 → frame 7
a_phys = 7 × 1024 + 452 = 7620
```

```
# (b) our convention
EAT = t_TLB + (2 − α) t_m
    = 10 + (2 − 0.9) × 100 = 120.0 ns
check: 0.9×110 + 0.1×210 = 99 + 21 = 120
115 = 10 + (2 − α)·100  ⇒  α = 0.95
```

Exam tips

- Write p, d, f on separate lines: partial credit.
- Sanity check: 0 ≤ d < page size; a_phys in the frame's range [7168, 8192).
- State the EAT convention you use.

FIFO, OPT and LRU with Three Frames

Reference string: 3, 5, 1, 0, 3, 1, 0, 3, 1, 4, 1, 0 · 3 frames · below: LRU (mint = page just loaded)

reference	**3**	**5**	**1**	**0**	**3**	**1**	**0**	**3**	**1**	**4**	**1**	**0**
frame 0	**3**	3	3	**0**	0	0	0	0	0	**4**	4	4
frame 1		**5**	5	5	**3**	3	3	3	3	3	3	**0**
frame 2			**1**	1	1	1	1	1	1	1	1	1
fault?	F	F	F	F	F					F		F

8
faults with FIFO

7
faults with LRU

5
faults with OPT (minimum)

Key decisions

- At 0 (4th ref): LRU evicts 3; OPT evicts 5 (never used again).
- At 4: OPT evicts 3 — also never needed again.

Short Calculations: EAT, Blocks and Quick Formulas

```
# t_m = 100 ns, t_fault = 10 ms, p = 0.0002
EAT = 0.9998 × 100 + 0.0002 × 10,000,000
    = 99.98 + 2,000 = 2099.98 ns   (×21)
# slow-down < 10 %:  100 + p(10^7 − 100) < 110
p < 10 / 9,999,900 ≈ 1.0 × 10⁻⁶
```

```
# record 75; 128-B records; 4096-B blocks
byte   = 75 × 128 = 9600
block  = ⌊9600 / 4096⌋ = 2
offset = 9600 − 2 × 4096 = 1408
last   = 9727 → block 2: fits
```

```
W1  U(n), p = 0.6, n = 3:      1 − 0.6^3 = 1 − 0.216 = 0.784
W1  batch, t_run 2, t_setup 3:  2 / (2 + 3) = 0.4
W3  overhead s = 0.1, q = 4:    0.1 / 4.1 = 2.44 %
W4  Amdahl S = 0.2, N = 4:      1 / (0.2 + 0.8/4) = 2.5  (limit 1/S = 5)
```

INTERMISSION

10-Minute Break

Next: the Linux half of the course — Weeks 9 to 14.

PART 2

Practical Recap: Weeks 9–14

All output on these slides is real, captured in a fresh ~/lab (bash mcmp117_lab_setup.sh).

Navigation, Files and Wildcards

```
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ../../beta/docs
student@okan-lab:~/lab/projects/beta/docs$ pwd
/home/student/lab/projects/beta/docs
student@okan-lab:~/lab/projects/beta/docs$ cd ~/lab
student@okan-lab:~/lab$ ls photos/img00?.jpg
photos/img001.jpg  photos/img004.jpg  photos/img007.jpg
photos/img002.jpg  photos/img005.jpg  photos/img008.jpg
photos/img003.jpg  photos/img006.jpg  photos/img009.jpg
student@okan-lab:~/lab$ echo photos/*.{png,JPG}
photos/img013.png photos/IMG_014.JPG
student@okan-lab:~/lab$ mkdir -p backup/{2025,2026}
student@okan-lab:~/lab$ cp docs/notes_0*.txt backup/2026/
student@okan-lab:~/lab$ ls backup/2026
notes_01.txt  notes_02.txt  notes_03.txt
```

Remember

- **Paths:** absolute starts with /; relative starts here; ~ home, . here, .. parent, cd - back.
- **Command anatomy:** command -options arguments.
- **Wildcards:** * any string, ? one character, [abc] one of; braces {a,b} {1..5}. The shell expands them, not the command.
- **Files:** mkdir -p, touch, cp -r, mv, rm -r (no recycle bin!), ln / ln -s.

Permissions: Reading and Setting the Mode



- **r = 4, w = 2, x = 1:** rwxr-x--- = 750.
- **Directories:** r list, w create/delete entries, x enter (cd).
- **Symbolic:** chmod u+x, go-w, a=r, g-x,o+r.
- **umask:** removes bits from 666 (files) / 777 (dirs): mode AND NOT umask; umask 027 → 640 / 750.
- **Root only:** chown, chgrp; sudo runs one command as root — think first.

```
student@okan-lab:~/lab$ id
uid=1015(student) gid=1017(student) groups=1017(student)
student@okan-lab:~/lab$ touch scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rw-rw-r-- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ chmod 750 scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rwxr-x--- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ chmod g-x,o+r scripts/run.sh
student@okan-lab:~/lab$ ls -l scripts/run.sh
-rwxr--r-- 1 student student 0 Oct 12 08:50 scripts/run.sh
student@okan-lab:~/lab$ umask
0002
```

Streams, Redirection and Pipes

The rules

- **fd 0 / 1 / 2:** stdin, stdout, stderr (Week 2: write(1, …)).
- **> file** overwrite, >> append, < input.
- **2> file** errors only; 2>&1 errors to where stdout goes; &> both; > /dev/null discard.
- **a | b:** stdout of a becomes stdin of b; the shell creates both processes (Week 3).
- **uniq -c** counts ADJACENT duplicates: sort first.

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log |
    sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
student@okan-lab:~/lab$ ls nosuchfile 2> errors.txt
student@okan-lab:~/lab$ cat errors.txt
ls: cannot access 'nosuchfile': No such file or directory
```

Processes, Jobs and Signals

Signal	No.	Meaning
SIGINT	2	Ctrl+C: interrupt
SIGTERM	15	kill default: please stop
SIGKILL	9	kill -9: cannot be caught
SIGSTOP	19	pause; cannot be caught
SIGCONT	18	continue (fg / bg)

- **Jobs:** cmd & (background), jobs, fg %1, bg %1, Ctrl+Z (SIGTSTP), kill %1.
- **Look:** ps, ps -ef, ps aux, pstree, top; STAT R S T Z (Week 3 states).
- **Priority:** nice −20 … 19; only root may lower it (Week 5).
- **$$** PID of the shell, $! last background PID.

```
student@okan-lab:~/lab$ sleep 600 &
student@okan-lab:~/lab$ echo $!
23441
student@okan-lab:~/lab$ jobs
[1]+  Running                 sleep 600 &
student@okan-lab:~/lab$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
23361 23360 Ss+  bash /home/student/w15/demo_recap.sh
23441 23361 S+   sleep 600
23445 23361 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ kill -l TERM KILL INT STOP CONT
15
9
2
19
18
```

Shell Scripting: the Building Blocks

```
#!/bin/bash
# skeleton of every exam script
if [ $# -ne 1 ]; then           # argument check
    echo "usage: $0 ARG" >&2
    exit 1
fi
count=0                         # no spaces at =
for f in "$1"/*; do             # loop over files
    if [ -f "$f" ]; then        # test: -f -d -e
        count=$((count + 1))    # arithmetic
    fi
done
echo "$count files in $1"
```

```
student@okan-lab:~/lab$ name=student; echo "Hello $name"
Hello student
student@okan-lab:~/lab$ echo 'Hello $name'
Hello $name
student@okan-lab:~/lab$ echo "2^10 = $((2 ** 10))"
2^10 = 1024
student@okan-lab:~/lab$ n=$(wc -l < data/students.csv);
    echo "students.csv has $n lines"
students.csv has 41 lines
student@okan-lab:~/lab$ [ -d logs ] && echo "logs is a
    directory"
logs is a directory
student@okan-lab:~/lab$ for c in 1 2 3; do echo "item $c";
    done
item 1
item 2
item 3
```

- Numbers: -eq -ne -lt -le -gt -ge. Strings: = != -z. Run: chmod +x s.sh; ./s.sh or bash s.sh. Debug: bash -x s.sh.

Navigation, Files and Permissions (Weeks 9–11)

Task	Command	Task	Command
Where am I?	pwd	Copy a folder	cp -r src/ dst/
List, long, hidden	ls -la (-h sizes)	Move / rename	mv old new
Go home / back / up	cd ~ cd - cd ..	Delete (asks)	rm -i f rm -r dir
Help	man ls ls --help	Links	ln f h ln -s f s
Make folders	mkdir -p a/b/c	Show file	cat less head tail -n
Empty file / time	touch f	Sizes	wc -l du -sh df -h
Who am I?	whoami id groups	Metadata	stat f file f ls -i
Permissions	chmod 640 f chmod u+x f	Owner (root)	sudo chown u:g f

Text, Pipes, Processes and Scripts (Weeks 12–14)

Task	Command	Task	Command
Find lines	grep -n -i -v -c -w	Processes	ps aux ps -ef top
Field k, delim d	cut -d, -f3	Background	cmd & jobs fg %1
Sort numerically	sort -n -r -t, -k5	Stop politely	kill PID (SIGTERM)
Count duplicates	sort | uniq -c	Force stop	kill -9 PID (SIGKILL)
Skip header line	tail -n +2 f	Lower priority	nice -n 10 cmd
Translate chars	tr a-z A-Z	Script start	#!/bin/bash
Save and show	cmd | tee out.txt	Variables	x=5; echo "$x"
Find files	find . -name "*.c" -type f	Arithmetic	$((x + 1)) $(cmd)

Write the Command: Permissions and umask

The question

- (a) report.sh: owner may read, write, run; group read and run; others nothing. Octal command?
- (b) After umask 027, which permissions get a new file and a new directory?

```
(a) rwx r-x --- = 4+2+1  4+0+1  0 = 750
    chmod 750 report.sh
(b) remove the umask bits (AND NOT 027):
    file 666 → 640  rw-r-----
    dir  777 → 750  rwxr-x---
```

```
student@okan-lab:~/lab$ touch report.sh
student@okan-lab:~/lab$ ls -l report.sh
-rw-rw-r-- 1 student student 0 Oct 12 08:50 report.sh
student@okan-lab:~/lab$ chmod 750 report.sh
student@okan-lab:~/lab$ ls -l report.sh
-rwxr-x--- 1 student student 0 Oct 12 08:50 report.sh
student@okan-lab:~/lab$ umask 027
student@okan-lab:~/lab$ touch new.txt; mkdir newdir
student@okan-lab:~/lab$ ls -ld new.txt newdir
-rw-r----- 1 student student    0 Oct 12 08:50 new.txt
drwxr-x--- 2 student student 4096 Oct 12 08:50 newdir
```

Write the Pipeline: Questions about access.log

The questions

- (a) Which three IP addresses sent the most requests?
- (b) How many requests returned status 404?
- (c) Which page caused the most 404 errors?
- **Fields** (space-separated): 1 = IP, 7 = page, 9 = status code.

```
student@okan-lab:~/lab$ cut -d' ' -f1 logs/access.log | sort |
    uniq -c | sort -rn | head -3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ cut -d' ' -f9 logs/access.log |
    grep -c '^404$'
57
student@okan-lab:~/lab$ grep ' 404 ' logs/access.log |
    cut -d' ' -f7 | sort | uniq -c | sort -rn | head -1
      9 /images/logo.png
```

Write the Script: Department Statistics

Write dept_stats.sh DEPT: print the number of students of DEPT in data/students.csv and their average final grade; without an argument print a usage message and exit with status 1.

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "usage: $0 DEPARTMENT" >&2
    exit 1
fi
dept=$1
count=0
sum=0
for f in $(grep ",$dept," data/students.csv | cut -d, -f5); do
    count=$((count + 1))
    sum=$((sum + f))
done
if [ $count -eq 0 ]; then
    echo "no students in $dept"
    exit 2
fi
echo "$dept: $count students, average final $((sum / count))"
```

```
student@okan-lab:~/lab$ ./scripts/dept_stats.sh ECT
ECT: 17 students, average final 63
student@okan-lab:~/lab$ ./scripts/dept_stats.sh CS
CS: 10 students, average final 65
student@okan-lab:~/lab$ ./scripts/dept_stats.sh
usage: ./scripts/dept_stats.sh DEPARTMENT
student@okan-lab:~/lab$ echo $?
1
```

- $((sum / count)) is integer division: 1072 / 17 = 63.06 → 63.
- The usage message goes to stderr (>&2).

AVOID THESE

Common Mistakes: Theory

Waiting vs. turnaround

Waiting = turnaround − burst. Do not forget the arrival time in turnaround.

Round Robin queue

A process arriving at the moment a quantum expires enters the queue BEFORE the preempted one (our convention).

Units

8 ms = 8,000,000 ns. Convert first, then substitute.

TLB miss

A miss costs two memory accesses (page table + data), plus the TLB lookup.

LRU vs. OPT

LRU looks backwards (least recently used); OPT looks forwards (used farthest in the future).

Block arithmetic

Records and blocks are numbered from 0. Check whether the last byte crosses a block boundary.

AVOID THESE

Common Mistakes: Commands and Scripts

uniq without sort

uniq -c only merges ADJACENT equal lines: always sort | uniq -c.

Spaces

x=5, not x = 5. [$a -gt 3], not [$a -gt 3].

Quotes

"$name" expands, '$name' does not. Quote file names with spaces: "final report.txt".

> vs >>

> overwrites the file, >> appends. 2> redirects only errors.

chmod 777, rm -r, kill -9

Too much permission, no undo, no clean-up. Use the gentlest tool that works.

Relative paths

A script run from another directory will not find data/...; run it from ~/lab or use absolute paths.

EXAM STRATEGY

How to Use the [90] Minutes

Before the exam

- Redo every worked example and review question of Weeks 1–7 and the lab sheets of Weeks 9–14.
- Solve the Practice Final in the study guide, then the sample paper — on paper, timed.
- Practise on Linux: type every command of the cheat sheet in ~/lab.
- Sleep. Tired brains make arithmetic mistakes.

During the exam

- Read the whole paper first (5 min); start with what you know best.
- Rough plan: about 1 minute per point, leave 10 minutes to check.
- Show every step: formula → numbers → result with unit.
- Commands: exact spelling and options; for scripts, logic first, details later.
- Never leave a question empty: partial credit exists.

One point per minute is a guide, not a rule: 100 points in [90] minutes leaves little slack — keep moving.

QUICK QUIZ

Check Your Understanding

1. P1 (0, 4), P2 (1, 2), P3 (2, 1) in ms, SRTF. What is the average waiting time?

2. Page size 4 KB. Logical address 0x3A7F: page number and offset? (in hex)

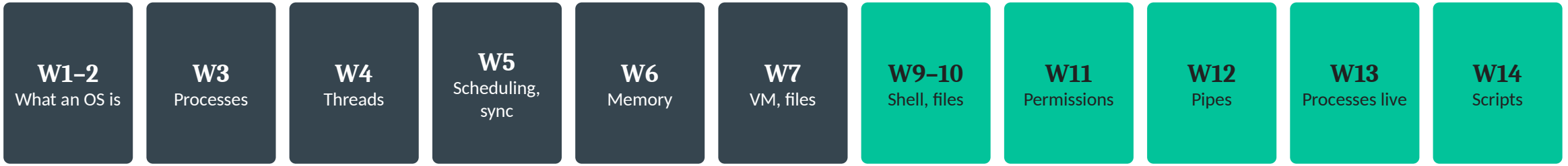
3. Which umask makes new files rw-r----- and new directories rwxr-x---?

4. Write a pipeline that prints the number of lines in logs/system.log that do NOT contain INFO.

The Course in Eight Lines

- **An OS** is a resource manager and an extended machine; dual mode and system calls protect it (W1–W2).
- **Processes** move between five states; the PCB and context switch make multitasking possible (W3).
- **Threads and schedulers** share the CPU: FCFS, SJF/SRTF, Priority, RR, judged by waiting and response time (W4–W5).
- **Shared data** needs mutual exclusion: mutex locks and semaphores (W5).
- **Memory** is translated by the MMU: paging, TLB, virtual memory, page replacement (W6–W7).
- **Files** are inodes with names in directories; links give a file several names (W7, W10).
- **The shell** turns commands into system calls and processes: navigation, files, permissions, pipes, jobs (W9–W13).
- **Scripts** automate everything you can type (W14).

What You Have Learned This Semester



You can explain

- why a computer needs an OS
- how processes share one CPU
- how memory is translated and paged

You can compute

- Gantt charts and waiting times
- addresses, EAT with TLB and page faults
- page-fault counts, block numbers

You can do

- work in any Linux shell
- secure files, analyse logs with pipelines
- manage processes, write scripts

Where to Go Next

Keep using Linux

- Install Ubuntu in a VM or WSL (wsl --install) and use it daily.
- A Raspberry Pi is a cheap Linux lab.
- Free practice: OverTheWire Bandit (a shell game).

Books (free and classic)

- OSTEP — Operating Systems: Three Easy Pieces (free online).
- Silberschatz et al., Operating System Concepts, 10th ed.
- W. Shotts, The Linux Command Line (free PDF).

Certifications

- LPI Linux Essentials (entry level).
- CompTIA Linux+ or LPIC-1 (next step).
- Later: Red Hat RHCSA, Linux Foundation LFCS. Check current versions and fees.

Useful sites: missing.csail.mit.edu (shell, tools) · explainshell.com (what does this command do?) · shellcheck.net (find bugs in scripts).

THANK YOU

Good Luck in the Final!

Questions before the exam? Ask on the course Telegram channel — everyone benefits from the answer. [Office hours before the exam: to be announced.]

Asst. Prof. Dr. Hasan Oğuz

Thank you for a great semester.
Keep the ~/lab folder and the course notes: they are a reference you can use long after the exam.